



Advanced Digital Design

PRESENTED BY:

Dr. Shasanka Sekhar Rout

Session 15 & 16

Lecture: Static Timing Analysis

- ✓ Need of Static Timing Analysis of Digital Circuits
- ✓ Metastability
- ✓ Setup and Hold Violations
- ✓ Time Borrowing and Time Stealing

Static Timing Analysis

Static Timing Analysis (STA):

- **Static Timing Analysis** is a method used in **VLSI design** to verify the **timing performance** of a digital circuit **without requiring simulation**.
- It ensures that signals propagate through the design **within required timing constraints**, avoiding setup and hold violations.

Why STA is Important?

- ✓ Ensures Timing Closure → Guarantees circuit works at the desired clock frequency.
- ✓ Faster than Simulation → Does not require input vectors like dynamic simulation.
- ✓ Detects Timing Violations → Finds setup and hold violations before fabrication.

Key Terminology:

1. **Clock Period** (T_{clk}) → Time between two consecutive rising edges.
2. **Setup Time** (T_{setup}) → Data must be stable before the clock edge.
3. **Hold Time** (T_{hold}) → Data must remain stable after the clock edge.
4. **Propagation Delay** (T_{pd}) → Time taken for a signal to propagate through a combinational path.
5. **Clock Skew** → Difference in arrival time of the clock signal at different registers.
6. **Clock Jitter** → Variation in clock period due to noise or process variations.

Static Timing Analysis

Basic Steps in STA:

1. Read Netlist & Constraints

- ✓ Load **Gate-Level Netlist** (from synthesis).
- ✓ Read **Timing Constraints** (from **SDC** file).

2. Extract Timing Paths

- ✓ Identify **clock-to-Q delay**, **combinational logic delay**, and **setup/hold times**.

3. Calculate Timing for Each Path

- ✓ Compute **arrival time** (T_{arr}) of signals.
- ✓ Compute **required time** (T_{req}) based on clock constraints.

4. Check for Violations

- ✓ **Setup violation** $\rightarrow T_{arr} > T_{req} - T_{setup}$ (Timing too slow).
- ✓ **Hold violation** $\rightarrow T_{arr} < T_{req} + T_{hold}$ (Data changing too soon).

5. Report and Fix Violations

- ✓ Modify **clock tree**, **buffer insertion**, or **optimize logic**.

Static Timing Analysis

Tools Used for STA:

- ◆ Synopsys PrimeTime
- ◆ Cadence Tempus
- ◆ OpenTimer (Open-source)
- ◆ Mentor Graphics Timing Analyzer

Fixing Timing Violations:

1. Setup Violation (Slow Path)

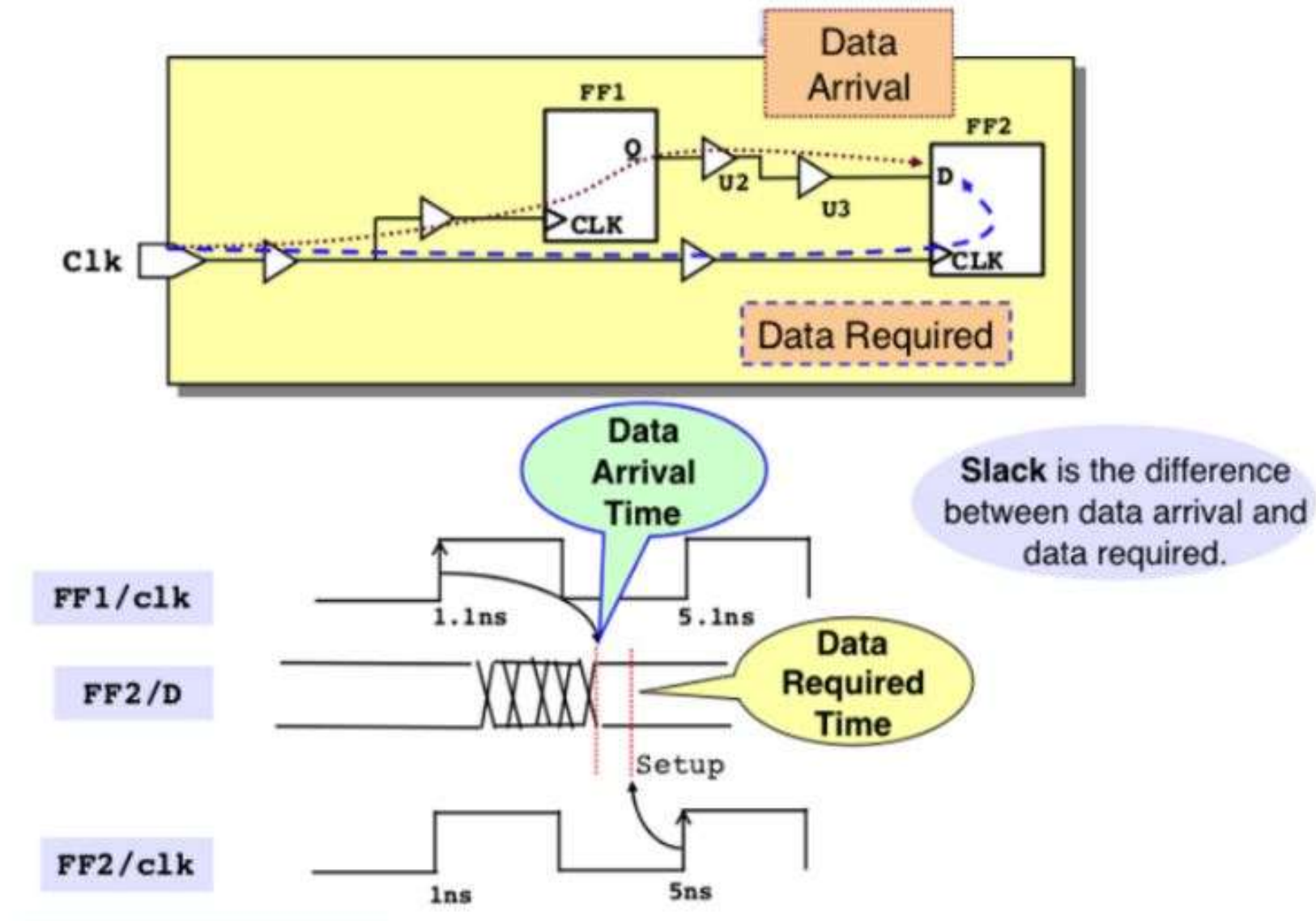
- ◆ Reduce Logic Delay → Optimize combinational logic.
- ◆ Insert Pipeline Registers → Reduce combinational path delay.
- ◆ Increase Clock Period → If possible, reduce frequency.

2. Hold Violation (Fast Path)

- ◆ Add Delay Buffers → Slow down fast data paths.
- ◆ Modify Clock Tree → Reduce clock skew.

Static Timing Analysis

Timing Report:



Static Timing Analysis

Timing Report:

report_timing

Header

```
Startpoint: FF1 (rising edge-triggered flip-flop clocked by Clk)
Endpoint: FF2 (rising edge-triggered flip-flop clocked by Clk)
Path Group: Clk
Path Type: max
```

Data arrival

Point	Incr	Path
clock Clk (rise edge)	0.00	0.00
clock network delay (propagated)	1.10 *	1.10
FF1/CLK (fdef1a15)	0.00	1.10 r
FF1/Q (fdef1a15)	0.50 *	1.60 r
U2/Y (buf1a27)	0.11 *	1.71 r
U3/Y (buf1a27)	0.11 *	1.82 r
FF2/D (fdef1a15)	0.05 *	1.87 r
data arrival time		1.87

Data required

clock Clk (rise edge)	4.00	4.00
clock network delay (propagated)	1.00 *	5.00
FF2/CLK (fdef1a15)		5.00 r
library setup time	-0.21 *	4.79
data required time		4.79

Slack

data required time	4.79
data arrival time	-1.87
slack (MET)	2.92

Static Timing Analysis

Launch Path

- Launch path is launch clock path which is responsible for launching the data at launch flip flop

Capture Path

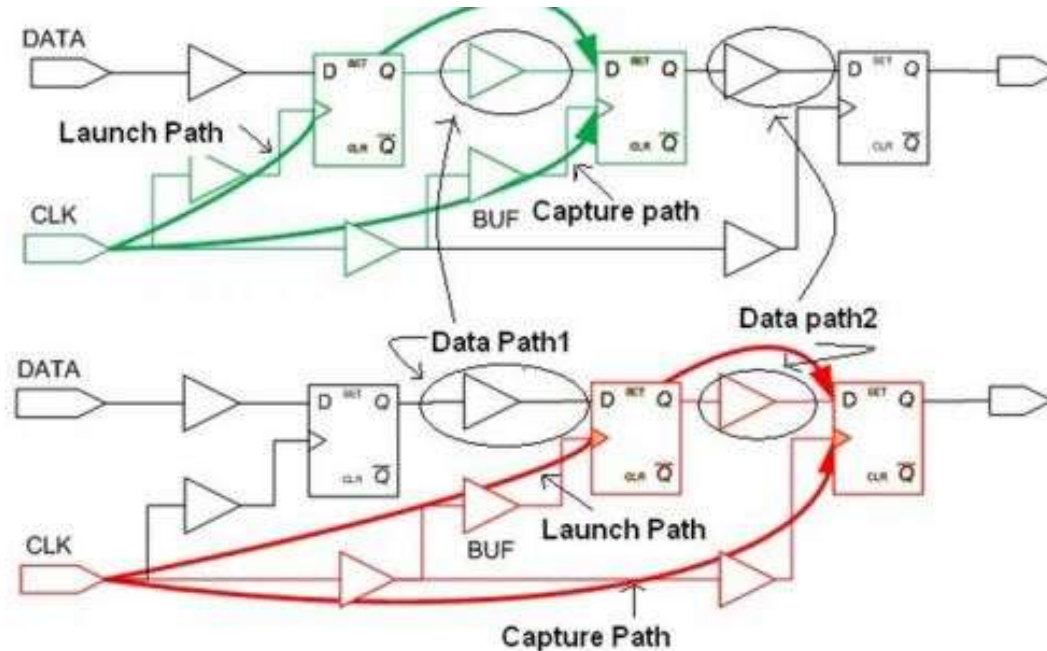
- Capture path is capture clock path which is responsible for capturing the data at capture flip flop

Arrival Time

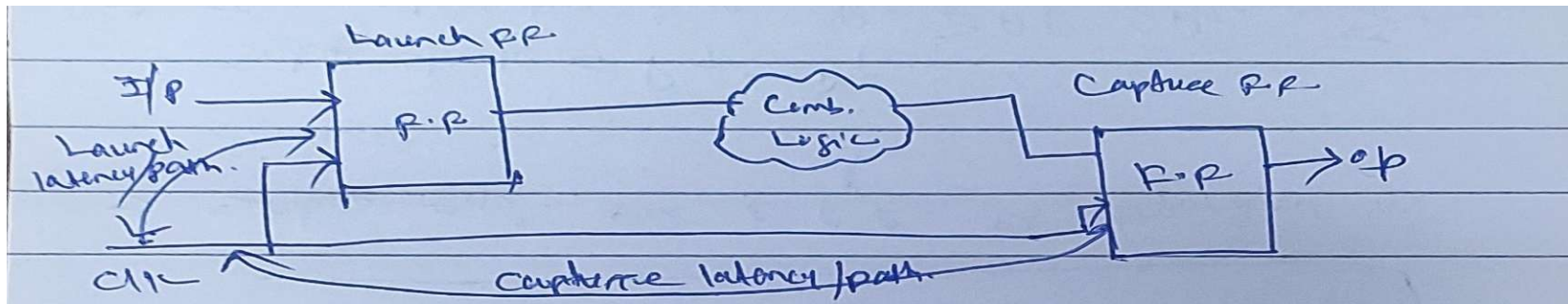
- Launch path and data path together constitute arrival time of data at the input of capture flip-flop

Required Time

- Capture clock period and its path delay together constitute required time of data at the input of capture register



Static Timing Analysis

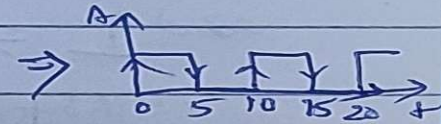


Sdc (a) Create - clock - name BCLK

- period 10

- waveform {0 5}

- [get - ports SCLK]



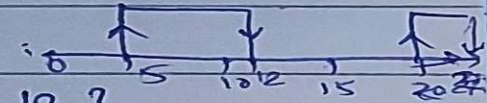
(b) Create - clock

- name S1SCLK

- period 15

- waveform {5 12}

- [get - ports ZAR]

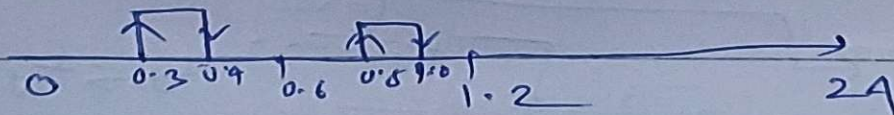


(c) Create - clk

- name JTAG

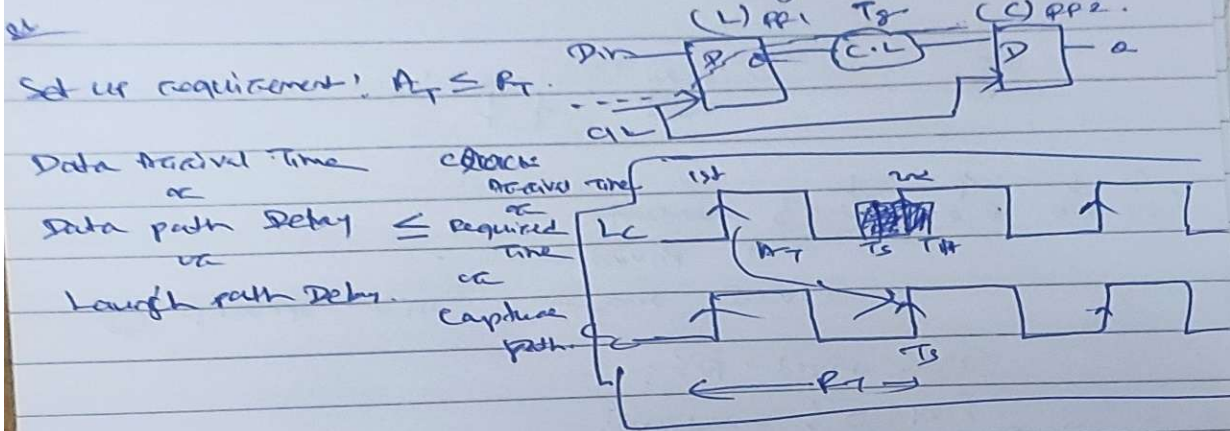
- period 1.2

- waveform {0.3 0.4 0.6 0.8 1.0}



Static Timing Analysis

② Find the indication where design meets setup required.



$$A_T = T_{\text{logic}} + T_{\text{clk-Q}} + T_g$$

$$R_T = T_{\text{capture}} + T_{\text{setup}} - T_{\text{hold}}$$

$$\Rightarrow T_L + T_{\text{clk-Q}} + T_g \leq T_C + T_{\text{pre}} - T_{\text{hold}}$$

$$\Rightarrow (T_L - T_C) + T_{\text{clk-Q}} + T_g + T_{\text{setup}} \leq T_{\text{pre}}$$

$$\Rightarrow \boxed{T_{\text{skew}} + T_{\text{clk-Q}} + T_g + T_{\text{setup}} \leq T_{\text{pre}}}$$

→ Setup eqn. for a FF. is dependant on frequency.

$$\text{Setup slack} = P_T - A_T$$

$$\rightarrow \text{To meet setup time } P_T \geq A_T \Rightarrow A_{T_{\text{max}}} \leq P_{T_{\text{min}}}$$

$$\therefore \text{Setup slack} = (P_{T_{\text{min}}} - A_{T_{\text{max}}})$$

→ Setup time depends on the max frequency.

Static Timing Analysis

SETUP TIME: The minimum amount of time the data should be stable before arrival of sensitive clock.

HOLD TIME: The minimum amount of time the data should be stable after arrival of sensitive clock.

✓ If launch clock has delay of 2 ps and capture clock has delay of 4 ps.

$$\text{skew} = 4 - 2 = 2 \text{ ps}$$

✓ **To meet set up time:**

$$R_T \geq A_T$$

$$A_{T_{\max}} \leq R_{T_{\min}}$$

$$(A_T = T_{\text{launch}} + T_{\text{Clk-Q}} + T_g) \quad (R_T = T_{\text{capture}} + T_{\text{period}} - T_{\text{setup}})$$

✓ **Set up slack** = $R_T - A_T$

✓ **To meet hold time:**

$$A_T \geq R_T$$

$$A_{T_{\min}} \geq R_{T_{\max}}$$

$$(A_T = T_{\text{launch}} + T_{\text{Clk-Q}} + T_g) \quad (R_T = T_{\text{capture}} + T_{\text{hold}})$$

✓ **Hold slack** = $A_T - R_T$

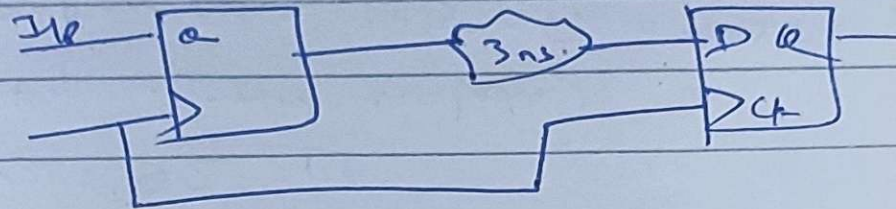
Static Timing Analysis

Problem 1

$$T_{\text{setup}} = 1 \text{ ns}$$

$$T_{\text{setup. hns}} = 0.5 \text{ ns}$$

$$T_{\text{read}} = 6 \text{ ns}$$



Analyse the circuit and do a waveform.

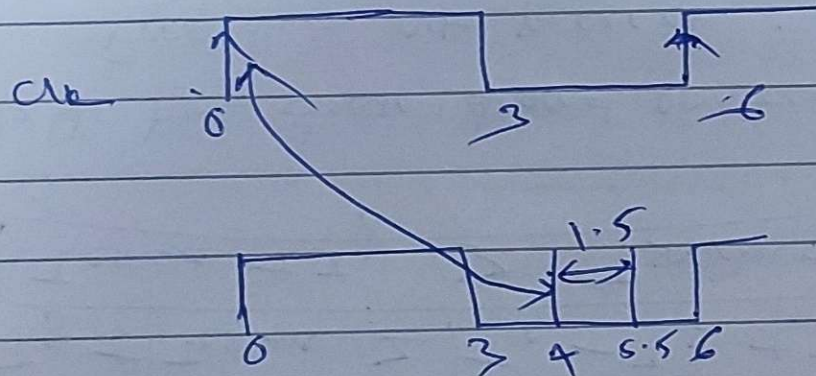
Soln $(A)_{\text{min}} = 1 + 3 = 4 \text{ ns}$

$$(R)_{\text{min}} = 6 - 0.5 = 5.5 \text{ ns}$$

$$\therefore (A)_{\text{min}} \leq (R)_{\text{min}} = 4 \leq 5.5$$

$\therefore$ So, setup time requirement is done.

$$\therefore \text{Setup slack} = R - A = 5.5 - 4 = 1.5 \text{ ns}$$



Static Time Analysis

Hold Analysis:

$$\rightarrow A_T \min \geq P_T \max$$

$$\text{Hold slack} = A_T - P_T$$

$$A_T = T_L + T_{ck-q} + T_g$$

$$P_T = T_c + T_{hold}$$

$$\rightarrow T_L + T_{ck-q} + T_g \geq T_c + T_{hold}$$

$$\rightarrow T_g + T_{ck-q} \geq [T_c - T_L] + T_{hold}$$

$$\rightarrow [T_{ck-q} + T_g] \geq T_{setup} + T_{hold}$$

[$\therefore$ No T_p , because same stage we are getting hold time]

$\rightarrow$ Hold time doesn't depend on frequency. $\therefore$

T_{setup} is not present.

Prob 4-1 Analyse the CN.

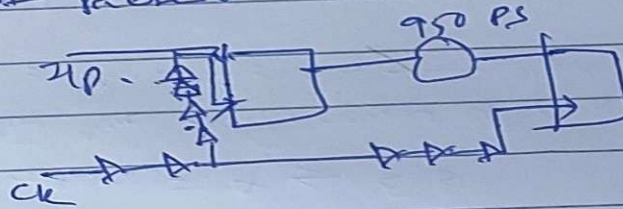
$$T_{ck-q} = 5 \text{ ps}$$

$$T_p = 1000 \text{ ps}$$

$$T_{setup} = 3 \text{ ps}$$

$$T_{huf} = 10 \text{ ps}$$

$$T_g = 950 \text{ ps}, T_H = 2 \text{ ps}$$



Sn

$$A_T = 60 + 5 + 950 = 1015 \text{ ps}$$

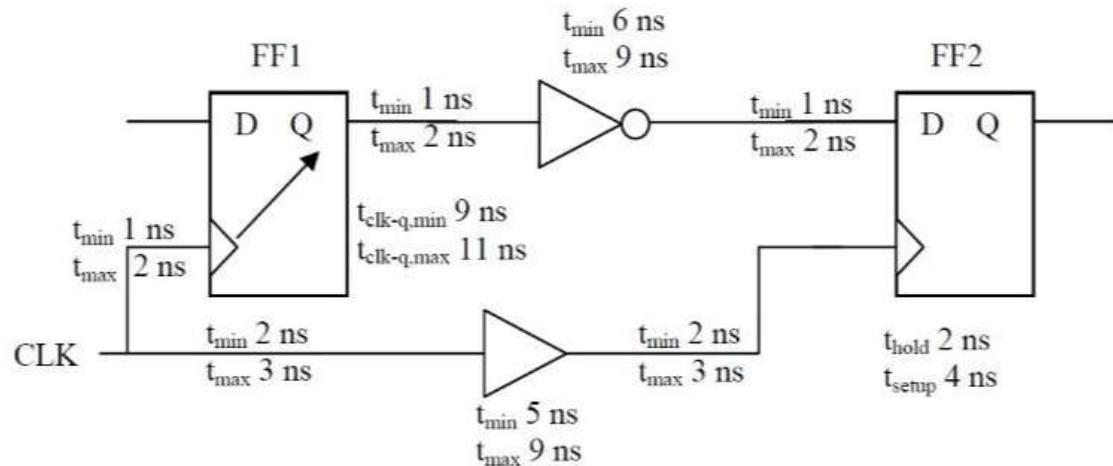
$$P_T = 50 + 2 = 52 \text{ ps}$$

$$\therefore A_T \geq P_T \text{ (Not violator)}$$

$$\text{Hold slack} \Rightarrow A_T - P_T = 1015 - 52 = 963 \text{ ps}$$

Static Time Analysis

STA Numerical: Analyze the following circuit for setup/hold check if setup time is 4 ns and time period is 15 ns.



Solution:

$$A_{T_{max}} = T_{lunch} + T_{clk_Q} + T_g = 2 + 11 + 2 + 9 + 2 = 26 \text{ ns}$$

$$R_{T_{min}} = T_{Cap} + T_{period} - T_{setup} = 2 + 5 + 2 + 15 - 4 = 20 \text{ ns}$$

Here $26 \text{ ns} > 20 \text{ ns}$ ($A_{T_{max}} \geq R_{T_{min}}$)

Setup requirement is not matching, so setup slack = $R_{T_{min}} - A_{T_{max}} = 20 - 26 = -6 \text{ ns}$

Static Time Analysis

Find maximum allowable delay for data path to meet setup, given lib tsetup = 0.08.

Example:

```
create_clock -period 2 [get_ports CLK]
set_clock_uncertainty -setup 0.14 [get_clocks CLK]
```



Ans: To meet setup, A_{Tmax} ≤ R_{Tmin}

$$\text{R}_{\text{Tmin}} = T_C + T_P - \text{t}_{\text{setup}} - \text{uncertainty} = T_C + 2 - 0.08 - 0.14 = T_C + 1.78$$

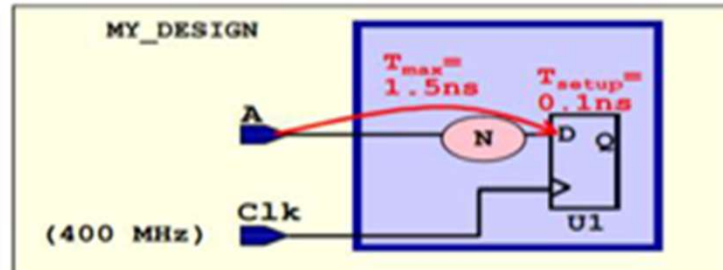
So, maximum allowable delay for data path to meet setup is $T_C + 1.78$ or 1.78

Static Time Analysis

How do you constrain MY_DESIGN for the indicated T_{max} ?

Spec:

Clock frequency = 400MHz. Maximum delay for path N = 1.5ns



Ans: Maximum delay for path N = 1.5 ns = $T_{max} = A_{Tmax}$

Clock frequency = 400 MHz, $T_P = 1/f = 1/400 \text{ MHz} = 2.5 \text{ ns}$

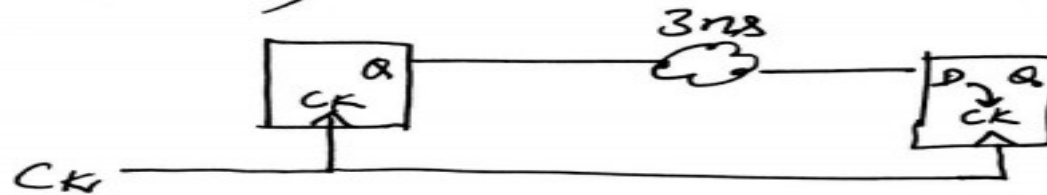
$R_{Tmin} = T_C + T_P - T_{setup} = 0 + 2.5 \text{ ns} - 0.1 \text{ ns} = 2.4 \text{ ns}$

Here 1.5 ns < 2.4 ns, so $A_{Tmax} \leq R_{Tmin}$. Setup requirement is matching for MY_DESIGN.

Setup slack = $R_{Tmin} - A_{Tmax} = 2.4 \text{ ns} - 1.5 \text{ ns} = 0.9 \text{ ns}$

Static Time Analysis

Ques: 5 Find Setup & Hold slack if it exists as a violation; also draw the waveform.



$\rightarrow CK_q = 1\text{ ns}$
 Setup time = 0.5 ns
 Clock period = 6 ns
 Hold time = 0.5 ns .

Ans: For Setup Analysis:

$$A_{Tmax} = T_{lunch} + T_{clk_Q} + T_g = 1 + 3 = 4\text{ ns}$$

$$R_{Tmin} = T_{Cap} + T_p - T_{setup} = 6 - 0.5 = 5.5\text{ ns}$$

Here $4\text{ ns} < 5.5\text{ ns}$ ($A_{Tmax} < R_{Tmin}$)

So, here setup is matching with setup slack = $5.5\text{ ns} - 4\text{ ns} = 1.5\text{ ns}$

For Hold Analysis:

$$A_{Tmin} = T_{lunch} + T_{clk_Q} + T_g = 1 + 3 = 4\text{ ns}$$

$$R_{Tmax} = T_{Cap} + T_{hold} = 0.5\text{ ns}$$

Here $4\text{ ns} > 0.5\text{ ns}$ ($A_{Tmin} > R_{Tmax}$)

Hold matching, so hold slack = $A_{Tmin} - R_{Tmax} = 4 - 0.5 = 3.5\text{ ns}$

Static Time Analysis

Setup Fixes:

- ✓ Buffer Insertion
- ✓ Upsizing The Driver Cell
- ✓ Reduce Net Length
- ✓ Cell Up Sizing.
- ✓ Drive Strength Of Launch Flop Increase.
- ✓ Logical Optimization On Data Path.
- ✓ Useful Skew.
- ✓ Pipelining.
- ✓ Use Sync Cells.
- ✓ Net Width Increase.
- ✓ Use Lvt Cells.
- ✓ Splitting The Combinational Logic.
- ✓ Increase Clock Period.
- ✓ Using Double Synchronizer Using Flip Flops.
- ✓ Redundant Via.
- ✓ Reduce The More Fanout Nets Within The Logic
- ✓ Double Via
- ✓ Layer Jumping

Hold Fixes:

- ✓ Delay Buffer Insertion.
- ✓ Cell Down Sizing.
- ✓ Increase Net Length.
- ✓ Use Hvt Cells.
- ✓ Scan Chain Reordering.
- ✓ Adjusting Timing Paths
- ✓ Can Be Fixed By Adding Delays On Input Ports.
- ✓ Clock Sizing
- ✓ One Can Add Look Up Latches
- ✓ Reduce Clock Skew
- ✓ Can Be Fixed By Adding Delays On Input ports
- ✓ Increase Netlength (Jog)

Static Timing Analysis

Metastability in Digital Circuits:

- ❖ Metastability occurs when a **flip-flop** or **register** enters an unstable state due to **violations of setup and hold time constraints**.
- ❖ Instead of settling into a definite **0** or **1**, the flip-flop remains in an **undefined** or oscillating state for a period before eventually resolving.
- ❖ Metastability is a major concern in **synchronous circuits**, especially when **asynchronous signals** (e.g., external inputs) interact with a clocked system.

Causes of Metastability:

- **Setup Time Violation**
 - Data changes too late before the clock edge.
- **Hold Time Violation**
 - Data changes too early after the clock edge.
- **Clock Domain Crossing (CDC)**
 - A signal moves from one clock domain to another with a different clock frequency.
- **Glitches or Noise**
 - Power fluctuations or signal integrity issues cause unstable transitions.

Static Timing Analysis

Metastability Resolution Time:

The time it takes for metastability to resolve is given by:

$$T_{meta} = \tau \cdot \ln \left(\frac{t_{setup} + t_{hold}}{t_0} \right)$$

τ = Time constant of the flip-flop.

t_{setup} , t_{hold} = Setup and hold time.

t_0 = Initial uncertainty window.

If T_{meta} is too long, the system may miss a clock cycle or produce incorrect outputs.

Effects of Metastability:

- ✗ **Data Corruption** → Unstable logic levels propagate.
- ✗ **Glitches & Timing Errors** → Unexpected transitions in combinational logic.
- ✗ **Increased Power Consumption** → Flip-flops oscillate, consuming more power.
- ✗ **Clock Domain Failures** → Cross-domain communication fails due to improper synchronization.

Static Timing Analysis

How to Prevent Metastability:

1. Use Synchronizer Flip-Flops

✓ A common technique is a 2-stage flip-flop synchronizer

FF1: Captures the asynchronous input (may go metastable).

FF2: Gives additional settling time, reducing metastability risk.

2. Use Proper Clock Domain Crossing (CDC) Techniques

If data moves between different clock domains:

- ◆ **Dual Flip-Flop Synchronization** (for single-bit signals).
- ◆ **FIFO Buffers** (for multi-bit signals).
- ◆ **Handshake Protocols** (for slow data transfer).

3. Increase Setup & Hold Margin

- ✓ **Optimize Timing** → Reduce delays in logic paths.
- ✓ **Adjust Clock Skew** → Ensure clocks reach flip-flops with minimal variation.
- ✓ **Use Faster Flip-Flops** → Reduces metastability resolution time τ .

Static Timing Analysis

Example: Metastability in Clock Domain Crossing:

If an external signal enters a system running at **100 MHz (10 ns clock period)** but violates the setup time, metastability may cause incorrect sampling.

Solution:

- ✓ Use a **2-stage synchronizer** → Allows flip-flop to settle.
- ✓ If data is multi-bit, use a **Gray code encoder** or **FIFO buffer**.

Summary:

- 📌 Metastability happens when flip-flops fail to meet setup/hold time.
- 📌 Clock domain crossings are the biggest source of metastability.
- 📌 Synchronization techniques like 2-stage flip-flops reduce the risk.
- 📌 Designing for proper timing closure (STA) helps avoid issues.

Static Timing Analysis

Time Borrowing and Time Stealing:

- ❖ **Time Borrowing** occurs in **level-sensitive (latch-based) designs**, where a **positive latch** can borrow time from the next clock cycle by allowing data to propagate while the latch is still transparent.
- ❖ **Time Stealing** happens in **edge-triggered flip-flop-based designs** when a **slower stage takes more time** than allocated, reducing the available time for the next stage.

Example of Time Borrowing:

- Consider two **positive latches** in a **pipeline**:

Latch1-->Combinational Logic(6ns)-->Latch2-->Combinational Logic(5ns)-->Latch 3

- If the **first stage needs 6 ns** (but the clock period is 5 ns), the extra **1 ns can be borrowed** from the second stage.
- The second stage now has **4 ns instead of 5 ns** to complete its operation.
- ✅ **Time borrowing ensures better pipeline balancing in latch-based systems.**

Static Timing Analysis

Example of Time Stealing:

- Consider two flip-flops (FF1 and FF2) with combinational logic between them:

FF1 --> Combinational Logic (7 ns) --> FF2 --> Combinational Logic (3 ns) --> FF3

- If the clock period is **5 ns**, the first path (FF1 to FF2) **exceeds its budget by 2 ns**.
- This **2 ns** is **stolen from the FF2 to FF3 path**, leaving it **only 3 ns** instead of 5 ns.
- ✗ If time stealing is excessive, the next stage may violate setup time.**

Feature	Time Borrowing (Latches)	Time Stealing (Flip-Flops)
Clock Type	Level-Sensitive (Latches)	Edge-Triggered (Flip-Flops)
Nature	Planned & Expected	Unintended & Risky
Effect	Improves timing flexibility	Can cause setup violations
Best Used In	Pipelined latch-based designs	Flip-flop-based synchronous circuits
Solution for Issues	Use latch-based designs with careful timing analysis	Use clock skew management & STA fixes

Static Timing Analysis

How to Handle Time Borrowing & Time Stealing?

✓ For Time Borrowing:

- Design circuits with **level-sensitive latches** instead of flip-flops.
- Use **timing-aware synthesis tools** to balance delays.

✓ For Time Stealing:

- **Clock skew optimization** → Adjust the clock arrival times.
- **Pipeline balancing** → Reduce long combinational delays in earlier stages.

THANK YOU

